

Graph-Constrained Reasoning with Symbolic Type Gates: The GCR Baseline and DCA-Trie Algorithms

Bernard Katamanso

August 2, 2026

1 Introduction

Large language models (LLMs) can reason over knowledge graphs (KGs) by generating natural language reasoning paths that traverse edges in the graph. *Graph-constrained decoding* (GCD) enforces this traversal by restricting the LLM’s vocabulary at each decode step to tokens that correspond to valid edges in the KG.

This technical report describes two algorithms for graph-constrained reasoning:

A1 GCR Baseline. Single-pass constrained decoding over a pre-built trie containing all valid paths up to L hops.

A2 DCA-Trie. Two variants that add symbolic type gates (from a TypeOracle) to constrain which edges enter the trie:

- (a) **v1 Static.** Pre-filters all paths through the TypeOracle before building the trie.
- (b) **v2 Iterative.** Hop-by-hop trie expansion with beam search, applying type gates at each step.

Both algorithms share the same underlying machinery—a trie that constrains the LLM’s token vocabulary—but differ in *which paths enter the trie* and *how many forward passes* are required.

2 Preliminaries

2.1 Knowledge Graph

Definition 1 (Knowledge Graph). *A knowledge graph is a tuple $\mathcal{G} = (V, E, \mathcal{R})$ where V is a set of entities, $\mathcal{R}$ is a set of relation types, and $E \subseteq V \times \mathcal{R} \times V$ is a set of labeled edges.*

Given a natural language question q , a KG subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ is extracted containing entities mentioned in q and their neighbours up to L hops. The question entities are $V_q \subseteq V$.

2.2 Reasoning Path

Definition 2 (Reasoning Path). *A reasoning path of length L through $\mathcal{G}$ is a chain*

$$c = (v_0, e_1, v_1, e_2, v_2, \dots, e_L, v_L)$$

where $v_0 \in V_q$ and $(v_{l-1}, e_l, v_l) \in E$ for all $l \in \{1, \dots, L\}$.

The terminal entity v_L is the candidate answer.

2.3 Trie

Definition 3 (Trie). A trie $\mathcal{T}$ is a prefix tree over token sequences. Given a tokenizer ϕ , each reasoning path c is encoded as a token sequence $\phi(c) = (t_1, t_2, \dots, t_n)$. The trie stores all such sequences and supports prefix-constrained generation: at each decode step with prefix p , the set of valid next tokens is

$$\mathcal{T}(p) = \{t \mid \exists \text{ sequence in } \mathcal{T} \text{ starting with } p \cdot t\}.$$

2.4 Constrained Decoding

Given a prompt $\mathbf{x}$ encoding the question q , an LLM produces token probabilities $P(t_l \mid t_{<l}, \mathbf{x})$ at each step l . Constrained decoding modifies this to

$$P_{\mathcal{T}}(t_l \mid t_{<l}, \mathbf{x}) = \begin{cases} \frac{P(t_l \mid t_{<l}, \mathbf{x})}{\sum_{t' \in \mathcal{T}(t_{<l})} P(t' \mid t_{<l}, \mathbf{x})} & \text{if } t_l \in \mathcal{T}(t_{<l}), \\ 0 & \text{otherwise.} \end{cases}$$

This is implemented via a `prefix_allowed_tokens_fn` callback that returns $\mathcal{T}(p)$ at each decode step, masking out all other tokens with $-\infty$ log-probability.

3 Algorithm 1: GCR Baseline

The GCR baseline (Luo et al., 2025) performs single-pass constrained decoding over a trie built from *all* reachable paths.

3.1 Procedure

1. **Subgraph extraction.** Extract $\mathcal{G}_q$ from $\mathcal{G}$ using the question entities V_q .
2. **Path enumeration.** Run depth-first search (DFS) from each $v \in V_q$ up to L hops, collecting all simple paths.
3. **Trie construction.** Tokenize each path string and insert into a MarisaTrie.
4. **Constrained generation.** Run the LLM with the trie constraint, generating the full reasoning path and answer in one forward pass.

Algorithm 1 GCR Baseline: Single-Pass Constrained Decoding

Require: question q , entities V_q , KG subgraph $\mathcal{G}_q$, max hops L , LLM $\mathcal{M}$, tokenizer ϕ

Ensure: reasoning path c^* , answer a

- 1: $\mathcal{P} \leftarrow \text{DFS}(\mathcal{G}_q, V_q, L)$ ▷ enumerate all paths
 - 2: $\mathcal{T} \leftarrow \text{BUILDTrie}(\phi, \mathcal{P})$ ▷ insert all path tokens
 - 3: $\mathbf{x} \leftarrow \text{PROMPT}(q)$ ▷ encode question as prompt
 - 4: $c^* \leftarrow \mathcal{M}.\text{GENERATE}(\mathbf{x}, \mathcal{T})$ ▷ single-pass constrained decoding
 - 5: $a \leftarrow \text{EXTRACTANSWER}(c^*)$
 - 6: **return** c^*, a
-

3.2 Properties

- **Completeness.** Every reachable path up to L hops is in the trie. The LLM can choose any of them.
- **Single forward pass.** The entire path is generated in one call to $\mathcal{M}.\text{GENERATE}$.

- **No semantic filtering.** All topologically valid paths are admitted, regardless of type compatibility.
- **Trie size.** $|\mathcal{P}| = O(d^L)$ paths, where d is the average out-degree. For typical KG subgraphs ($d \approx 20$, $L = 2$), this yields thousands of paths.

4 Algorithm 2: DCA-Trie

DCA-Trie adds a *TypeOracle* that applies symbolic type gates to filter paths before they enter the trie. The key insight: not all topologically valid paths are semantically coherent.

4.1 The TypeOracle

The TypeOracle is a purely symbolic component that extracts type information from the KG’s ontology schema (Freebase `common.topic.notable_types`, `rdf-schema#range`, etc.). It provides two gates:

Definition 4 (Range Gate). *Given a relation $e \in \mathcal{R}$ and a tail entity v , the range gate checks whether v ’s types are compatible with e ’s declared range:*

$$\text{rangeGATE}(e, v) \iff \mathcal{T}(v) \cap \text{range}(e) \neq \emptyset \vee \mathcal{T}(v) = \emptyset \vee \text{range}(e) = \emptyset.$$

The empty-set disjuncts make the gate conservative: if types are unknown, the path is admitted.

Definition 5 (Type Gate). *Given a candidate terminal entity v , expected answer types $A \subseteq \text{Types}$, hop index l , and max hops L , the type gate checks:*

$$\text{TYPEGATE}(v, A, l, L) \iff l < L \vee A = \emptyset \vee \mathcal{T}(v) = \emptyset \vee \mathcal{T}(v) \cap A \neq \emptyset.$$

At intermediate hops ($l < L$), the gate always passes. At the terminal hop, it checks type compatibility with the expected answer types.

Definition 6 (Admissibility). *A candidate edge (e_l, v_l) is admissible iff it passes both gates:*

$$\text{admissible}(e_l, v_l) = \text{rangeGATE}(e_l, v_l) \wedge \text{TYPEGATE}(v_l, A, l, L).$$

4.2 Answer Type Inference

The expected answer types A are inferred from the question string via pattern matching:

$$A = \bigcup_{\text{pattern}(q) \rightarrow T_k} T_k$$

where T_k are pre-defined type sets (e.g., “*who*” $\rightarrow$ PersonTypes, “*where*” $\rightarrow$ LocationTypes). If no pattern matches, $A = \emptyset$ (unconstrained), and the type gate passes everything.

4.3 Algorithm 2a: DCA-Trie v1 (Static)

v1 is a direct refinement of the GCR baseline: filter paths through the TypeOracle *before* building the trie.

Algorithm 2 DCA-Trie v1: Static Type-Oracle Filtering

Require: question q , entities V_q , KG subgraph $\mathcal{G}_q$, max hops L , LLM $\mathcal{M}$, tokenizer ϕ , TypeOracle Ω

Ensure: reasoning path c^* , answer a

```
1:  $\mathcal{P} \leftarrow \text{DFS}(\mathcal{G}_q, V_q, L)$  ▷ enumerate all paths
2:  $A \leftarrow \Omega.\text{INFERENCEANSWERTYPES}(q)$ 
3: if  $A = \emptyset$  then
4:    $A \leftarrow \Omega.\text{INFERENCEFROMPATHS}(\mathcal{P})$  ▷ fallback: use terminal entity types
5: end if
6:  $\mathcal{P}_{\text{filt}} \leftarrow \{\}$ 
7: for  $c = (v_0, e_1, v_1, \dots, e_L, v_L) \in \mathcal{P}$  do
8:   if  $\forall l : \text{rangeGATE}(e_l, v_l)$  and  $\text{TYPEGATE}(v_L, A, L, L)$  then
9:      $\mathcal{P}_{\text{filt}} \leftarrow \mathcal{P}_{\text{filt}} \cup \{c\}$ 
10:   end if
11: end for
12:  $\mathcal{T} \leftarrow \text{BUILDTrie}(\phi, \mathcal{P}_{\text{filt}})$ 
13:  $\mathbf{x} \leftarrow \text{PROMPT}(q)$ 
14:  $c^* \leftarrow \mathcal{M}.\text{GENERATE}(\mathbf{x}, \mathcal{T})$ 
15:  $a \leftarrow \text{EXTRACTANSWER}(c^*)$ 
16: return  $c^*, a$ 
```

Properties.

- **Same single forward pass** as the baseline.
- **Smaller trie.** Filtering removes type-incompatible paths, typically reducing trie size by 25% (WebQSP) to 40% (CWQ).
- **Risk: over-filtering.** If the TypeOracle is too aggressive, correct paths are removed and the answer becomes unreachable. This is measured by the *false negative rate* (FNR) of the oracle.

4.4 Algorithm 2b: DCA-Trie v2 (Iterative Beam Search)

v2 replaces the single-pass approach with hop-by-hop trie expansion. At each hop, the trie is rebuilt from the 1-hop neighbours of the current beam’s head pool, filtered through the TypeOracle.

Definition 7 (Head Pool). *For a beam b at step l , the head pool $H_b^{(l)} \subseteq V$ is the set of entities that are available as starting points for the next hop. Initially $H_b^{(0)} = V_q$. After each hop, the new entity v_l is added: $H_b^{(l)} = H_b^{(l-1)} \cup \{v_l\}$.*

Definition 8 (Beam Unit). *A beam unit is a tuple $b = (\sigma, H_b, s, l)$ where:*

- σ is the accumulated prompt (question + generated path so far),
- H_b is the head pool,
- s is the cumulative log-probability score,
- l is the current hop index.

Algorithm 3 DCA-Trie v2: Iterative Beam Search with Type-Oracle Gates

Require: question q , entities V_q , KG subgraph $\mathcal{G}_q$, max hops L , LLM $\mathcal{M}$, tokenizer ϕ , TypeOracle Ω , beam size B , max new tokens N

Ensure: reasoning path c^* , answer a

```
1:  $A \leftarrow \Omega.\text{INFERANSWERTYPES}(q)$ 
2:  $\mathbf{x} \leftarrow \text{PROMPT}(q)$ 
3:  $\mathcal{B}_0 \leftarrow \{\}$ 
4: for  $v \in V_q$  do
5:    $\mathcal{P}_1 \leftarrow \text{GATEDPATHS}(\mathcal{G}_q, v, \Omega, A, 1, L)$ 
6:   if  $\mathcal{P}_1 \neq \emptyset$  then
7:      $\mathcal{B}_0 \leftarrow \mathcal{B}_0 \cup \{(\mathbf{x}, \{v\}, 0, 0)\}$ 
8:   end if
9: end for
10: if  $\mathcal{B}_0 = \emptyset$  then return NONE
11: end if
12:  $\mathcal{B} \leftarrow \mathcal{B}_0$ 
13: for  $l = 1$  to  $L$  do
14:    $\mathcal{B}' \leftarrow \{\}$ 
15:   for beam  $b = (\sigma, H_b, s, l-1) \in \mathcal{B}$  do
16:      $\mathcal{P}_l \leftarrow \{\}$ 
17:     for  $h \in H_b$  do
18:        $\mathcal{P}_l \leftarrow \mathcal{P}_l \cup \text{GATEDPATHS}(\mathcal{G}_q, h, \Omega, A, l, L)$ 
19:     end for
20:     if  $\mathcal{P}_l = \emptyset$  then continue
21:     end if
22:      $\mathcal{T}_b \leftarrow \text{BUILDTRIE}(\phi, \mathcal{P}_l)$ 
23:      $\sigma' \leftarrow \sigma \parallel \langle \text{PATH} \rangle$ 
24:      $\text{RESULTS} \leftarrow \mathcal{M}.\text{GENERATE}(\sigma', \mathcal{T}_b, \text{num\_beams} = B, \text{return\_scores} = \text{true})$ 
25:     for  $(\text{path}_i, s_i) \in \text{RESULTS}$  do
26:        $v_l \leftarrow \text{LASTENTITY}(\text{path}_i)$ 
27:        $\sigma'' \leftarrow \sigma \parallel \text{path}_i$ 
28:        $\mathcal{B}' \leftarrow \mathcal{B}' \cup \{(\sigma'', H_b \cup \{v_l\}, s + s_i, l)\}$ 
29:     end for
30:   end for
31:    $\mathcal{B} \leftarrow \text{TOPB}(\mathcal{B}', B)$   $\triangleright$  keep best  $B$  beams by score
32: end for
33:  $b^* \leftarrow \arg \max_{b \in \mathcal{B}} b.s$ 
34:  $c^* \leftarrow \text{PARSEPATH}(b^*. \sigma)$ 
35:  $a \leftarrow \text{EXTRACTANSWER}(c^*)$ 
36: return  $c^*, a$ 
```

Algorithm 4 GatedPaths: 1-Hop Enumerate + Type-Oracle Filter

Require: head entity h , KG $\mathcal{G}_q$, TypeOracle Ω , answer types A , hop l , max hops L

Ensure: list of allowed 1-hop path strings

```
1:  $\mathcal{P} \leftarrow \{\}$ 
2: for  $(h, e, v) \in \mathcal{G}_q.\text{NEIGHBORS}(h)$  do
3:   if  $\text{rangeGATE}(e, v)$  and  $(l < L \text{ or } \text{TYPEGATE}(v, A, l, L))$  then
4:      $\mathcal{P} \leftarrow \mathcal{P} \cup \{h \rightarrow e \rightarrow v\}$ 
5:   end if
6: end for
7: return  $\mathcal{P}$ 
```

The GatedPaths subroutine.

Properties.

- **Multiple forward passes.** Each hop requires a separate call to $\mathcal{M}.\text{GENERATE}$. With L hops and B beams, worst case is $L \times B$ forward passes.
- **Adaptive trie size.** The trie is rebuilt at each hop from only the 1-hop neighbours of the current head pool, so it is much smaller than the baseline’s full-path trie.
- **Beam search.** Multiple candidate paths are explored simultaneously, with scores accumulated across hops.
- **Backtracking.** If a beam’s head pool yields no valid paths, the beam is dropped (not expanded further).

5 Complexity Analysis

5.1 Path Explosion

The number of simple paths of length L from a set of start entities in a graph with average out-degree d is

$$|\mathcal{P}| = O(|V_q| \cdot d^L).$$

For WebQSP ($|V_q| \approx 1$, $d \approx 20$, $L = 2$): $|\mathcal{P}| \approx 400$ paths. For CWQ ($d \approx 30$, $L = 3$): $|\mathcal{P}| \approx 27,000$ paths.

5.2 Comparison

Table 1: Complexity comparison of the three algorithms.

	GCR Baseline	DCA-Trie v1	DCA-Trie v2
DFS passes	1	1	0
Trie builds	1	1	L
Model forward passes	1	1	$\leq L \times B$
Trie size (paths)	$O(d^L)$	$O(d^L)$	$O(d)$
Type gates applied	None	All at once	Per hop

Trie size. The baseline and v1 build one trie from all paths ($O(d^L)$ entries). v2 builds a new trie at each hop from only 1-hop neighbours ($O(d)$ entries per hop), but does so L times.

Model passes. The dominant cost is the LLM forward pass (≈ 6 s on an A100 for an 8B model). Baseline and v1 each require one pass. v2 requires up to $L \times B$ passes (one per beam per hop), but in practice many beams are pruned early.

When v2 wins. v2 is advantageous when:

- The graph is large (d^L is huge), so the baseline’s trie is unwieldy.
- Type gates are aggressive, so filtering at each hop prunes more effectively than pre-filtering all paths.
- Beam search finds better paths than the baseline’s single greedy/group-beam pass.

When baseline/v1 win. The baseline or v1 is advantageous when:

- The graph is small (d^L is manageable).
- Latency matters (one forward pass vs. many).
- Type gates are conservative (few paths removed), so the filtering overhead isn’t justified.

6 Relationship Between the Algorithms

The three algorithms form a hierarchy:

$$\text{GCR Baseline} \supset \text{DCA-Trie v1} \subset \text{DCA-Trie v2}$$

- **v1 $\subset$ Baseline.** v1’s trie is a subset of the baseline’s trie (filtered paths $\subseteq$ all paths). v1 generates from a smaller constraint set.
- **v2 is not a subset of either.** v2’s per-hop trie expansion explores a different set of paths than the baseline’s full DFS. Because v2 only enumerates 1-hop neighbours at each step (not all L -hop paths upfront), it can reach entities that the DFS might miss due to path explosion cutoffs.
- **All three use the same trie mechanism.** The `prefix_allowed_tokens_fn` callback is identical; only the trie’s contents differ.

7 Discussion

7.1 The Role of the TypeOracle

The TypeOracle is *not* a learned component. It is a purely symbolic rule engine that extracts type information from the KG’s ontology schema. Its two gates enforce:

1. **Local consistency** (range gate): the tail entity’s type must be compatible with the relation’s declared range.
2. **Global consistency** (type gate): the terminal entity’s type must be compatible with the expected answer types.

The conservative default (admit when types are unknown) means the oracle never over-filters when information is missing. Its effectiveness depends on the quality and coverage of the ontology schema.

7.2 Limitations

- **Ontology coverage.** The TypeOracle’s hand-curated schema covers ~ 38 Freebase relations. Auto-mining from data triples extends coverage, but mined ranges may be noisy.
- **Evaluation metric.** The standard Hits@1 metric uses substring matching (`answer` $\in$ `prediction`), which is lenient. Exact-match evaluation would give lower but more honest numbers.
- **v2 latency.** Multiple forward passes make v2 slower than the single-pass baseline, despite smaller tries.

Acknowledgements

This work builds on the GCR framework (Luo et al., 2025) and the DoG decoding paradigm (Lin et al., 2025).